How to Continue: GS1 DataMatrix Label System

What This Project Is

A web application for creating, previewing, and printing GS1 DataMatrix UDI labels for medical devices. It complies with FDA UDI and EU MDR regulations. The app runs on a local PC with a Zebra GK420D USB printer and is accessible to up to 6 users on the same office LAN.

What Was Completed in the Previous Chat

Over the course of the conversation, we:

1. **Designed the full system architecture** — React + Node.js + JSON file storage (no SQL database), with Zebra Browser Print for USB printing
2. **Defined the critical business rule** — LOT mode labels have NO expiration date; SERIAL mode labels INCLUDE expiration date. The HRI (human-readable) text order is: GTIN → Identifier → Date(s), which differs from the DataMatrix encoding order.
3. **Created 5 detailed Claude Code agent prompts:**
   - **Agent 1** (Phases 1–3): Project foundation, JSON storage, auth, GS1 barcode engine, label data form with live preview
   - **Agent 2** (Phases 4–5): Label template designer (react-konva), template import/export (.gslabel files), ZPL II generation, PNG/PDF export
   - **Agent 3** (Phases 6–8): Zebra printer connection, batch/sequential printing, LAN print queue, audit log UI, deployment
   - **Agent 4** (QA): Frontend & UX testing with 100+ test cases
   - **Agent 5** (QA): Backend, API, GS1 encoding, and ZPL correctness testing
4. **Created a Master Plan** summarizing the entire project
5. **Built an interactive UI preview** with real GS1 DataMatrix encoding (bwip-js), responsive label shapes, and all major pages

Key Decisions Made

- **JSON files instead of SQL** — max 6 users, stored in `/data/*.json` with file locking
- **LAN-only deployment** — server binds to 0.0.0.0, users access via local IP
- **LAN print queue** — Zebra Browser Print only works on localhost, so LAN users queue jobs and the printer PC operator processes them
- **Variable label sizes** — square or rectangle, mm or inches, with preset library

- **Template import/export** — .gslabel and .gslabel-pack file formats for the ~20 label types
- **PNG/PDF export** — single labels and batch export (up to 500 labels per batch)
- **HRI display order** — (01) GTIN → identifier (lot or serial) → dates — differs from encoding order
- **No expiration in lot mode** — enforced at every layer (form, encoding, ZPL, HRI, preview, print)

Artifacts to Paste into This New Chat

Paste these artifacts in this order so I have full context:

1. **Master Plan** — the project overview and architecture summary
2. **Agent 1 prompt** — to review or modify Phase 1–3 work
3. **Agent 2 prompt** — to review or modify Phase 4–5 work
4. **Agent 3 prompt** — to review or modify Phase 6–8 work
5. **Agent 4 prompt** — QA frontend testing
6. **Agent 5 prompt** — QA backend testing

How to Use the Agent Prompts in Claude Code

For building:

1. Open a new Claude Code session
2. Paste **Agent 1's prompt** in full
3. Let it build Phases 1–3, run the test checklist
4. Open a new Claude Code session
5. Tell it: "Read the existing codebase first, then follow this prompt:" and paste **Agent 2's prompt**
6. Repeat for **Agent 3**

For QA:

1. After all 3 build agents complete, open a new Claude Code session
2. Paste **Agent 4's prompt** — it will test the frontend and produce a bug report
3. Open another session, paste **Agent 5's prompt** — it will test the backend
4. Collect both bug reports, fix the issues, re-run QA

Tips:

- Each agent prompt is self-contained — it includes the full spec for its phases
- Build agents include test checklists at the end of each phase

- QA agents produce structured bug reports with severity levels
- If an agent needs to know what already exists, tell it to read the project files first
- The prompts reference exact file paths, TypeScript interfaces, and function signatures to minimize ambiguity

What to Ask Me in This New Chat

- "Review the agent prompts and suggest improvements"
- "I ran Agent 1 and got [error] — help me fix it"
- "Agent 4 found these bugs: [paste report] — update the prompts to fix them"
- "Add a new feature: [describe it]"
- "Modify the template designer to support [X]"
- "Update the UI preview to include [X]"